

Chapter 14 Graph

GRAPHS

◆ Prompt

What is a graph and why do we use it?

✓ Answer

A **graph** is a mathematical structure used to represent **relationships** between objects.

- Objects $\rightarrow$ **Vertices (nodes)**
- Relationships $\rightarrow$ **Edges (links)**

Formally:

$G = (V, E)$

V = set of vertices

E = collection of edges connecting vertices

★ Graphs are used in:

- Maps & navigation
- Computer networks
- Social networks
- Transportation systems
- Electrical & plumbing networks

✎ Simple Diagram

A — B

| |

C — D

Vertices: {A, B, C, D}

Edges: { (A,B) , (A,C) , (B,D) , (C,D) }

◆ Prompt

What are vertices and edges?

✓ Answer

- **Vertex (plural: vertices)** → an object (city, computer, person)
- **Edge** → a connection between two vertices

◆ Prompt

What is the difference between directed and undirected graphs?

✓ Answer

◆ Undirected Graph

- Edge has **no direction**
- $(u, v) = (v, u)$

Example: friendship, co-authorship

A — B

◆ Directed Graph (Digraph)

- Edge has a **direction**
- $(u \rightarrow v) \neq (v \rightarrow u)$

Example: flights, inheritance

A → B

◆ Mixed Graph

Contains **both** directed and undirected edges.

Example: city map (one-way + two-way streets)

◆ Prompt

What does adjacent and incident mean?

✓ Answer

- **Adjacent vertices:** two vertices connected by an edge
- **Incident edge:** an edge connected to a vertex

A — B

- A and B are adjacent
- Edge (A,B) is incident to A and B

◆ Prompt

What is degree of a vertex?

✓ Answer

◆ Undirected Graph

- **Degree** = number of incident edges

A — B — C

$\deg(B) = 2$

◆ Directed Graph

- **In-degree**: incoming edges
- **Out-degree**: outgoing edges

A → B → C

↑

D

- $\text{indeg}(B) = 2$
- $\text{outdeg}(B) = 1$

◆ Prompt

How do graphs model real-world systems?

✓ Answer

System	Vertices	Edges
City map	Intersections	Roads
Internet	Computers	Connections
Flights	Airports	Flights
Programs	Classes	Inheritance

◆ Prompt

What are parallel edges and self-loops?

✓ Answer

◆ Parallel Edges

More than one edge between same vertices

$A \rightleftarrows B$

$A \rightleftarrows B$

Example: multiple flights between two cities

◆ Self-Loop

Edge from a vertex to itself

$A \rightarrow A$

◆ Simple Graph

A graph with:

- ✗ No parallel edges
- ✗ No self-loops

★ Most graphs studied are simple graphs

◆ Prompt

What is a path and a cycle?

✓ Answer

◆ Path

A sequence of vertices connected by edges

$A \rightarrow B \rightarrow C \rightarrow D$

◆ Simple Path

No vertex repeats

◆ Cycle

Path that starts and ends at same vertex

$A \rightarrow B \rightarrow C \rightarrow A$

◆ Directed Cycle

All edges follow direction

◆ Prompt

What does it mean for a vertex to reach another?

✓ Answer

Vertex **u reaches v** if there is a path from u to v.

◆ Connected Graph

Every pair of vertices has a path between them

$A \text{ --- } B \text{ --- } C$

◆ Strongly Connected (Directed)

For all u, v:

- u reaches v
- v reaches u

$A \rightarrow B \rightarrow C$



◆ Prompt

What is a subgraph?

✓ Answer

A **subgraph** contains:

- A subset of vertices
- A subset of edges

◆ Connected Components

Maximal connected subgraphs

A — B C — D

Two components: $\{A, B\}$ and $\{C, D\}$

◆ Prompt

What are trees and forests in graph theory?

✓ Answer

🌳 Tree

- Connected
- No cycles
- n vertices $\rightarrow n-1$ edges

A — B — C

🌲 Forest

- Collection of trees
- No cycles
- Not necessarily connected

🌳 Spanning Tree

- Subgraph
- Includes all vertices
- Is a tree

✦ Proposition 14.8

Sum of degrees in an undirected graph

$$\sum \deg(v) = 2m$$

Why?

Each edge is counted **twice** (once per endpoint).

✦ Proposition 14.9 (Directed Graph)

$$\sum \text{indeg}(v) = \sum \text{outdeg}(v) = m$$

Each edge contributes:

- 1 to in-degree
- 1 to out-degree

✦ Proposition 14.10 (Max Edges)

Graph Type	Maximum edges
Undirected	$n(n-1)/2$
Directed	$n(n-1)$

✦ Proposition 14.11 (Trees & Graphs)

Graph Type	Edges
Connected	$\geq n-1$
Tree	$= n-1$
Forest	$\leq n-1$

THE GRAPH ADT (Abstract Data Type)

◆ Prompt

What is the Graph ADT and why do we need it?

✓ Answer

The **Graph ADT (Abstract Data Type)** defines **what operations** can be performed on a graph **without specifying how it is implemented**.

A graph is modeled using **three data types**:

1. **Vertex**
2. **Edge**
3. **Graph**

This separation:

- Keeps graph design clean
- Hides internal representation

- Makes algorithms independent of implementation

◆ Prompt

What is a Vertex in the Graph ADT?

✓ Answer

A **Vertex** is a lightweight object that:

- Represents a node in the graph
- Stores **one user-defined element** (data)

✦ Example elements:

- Airport code (JFK, LAX)
- City name
- Person ID

◆ Vertex Method

`element()`

➡ Returns the data stored in the vertex

◆ Prompt

What is an Edge in the Graph ADT?

✓ Answer

An **Edge** represents a connection between two vertices and:

- Stores an element (weight, label, cost, flight number)
- Knows which vertices it connects

◆ Edge Methods

1. `element()`

Returns the value stored in the edge
(e.g., distance, cost, flight number)

2. `endpoints()`

Returns a tuple:

`(u, v)`

- u = origin
- v = destination
(orientation is arbitrary in undirected graphs)

3. *opposite(v)*

Given one endpoint v , returns the **other endpoint**

Diagram

$(u) \text{ ----[Edge: 500km]----> } (v)$

- $\text{endpoints}() \rightarrow (u, v)$
- $\text{opposite}(u) \rightarrow v$
- $\text{opposite}(v) \rightarrow u$

◆ Prompt

What does the Graph ADT represent?

✓ Answer

The **Graph ADT** represents the entire structure:

- Collection of vertices
- Collection of edges
- Can be **directed or undirected**

✦ Mixed graphs are handled as directed graphs by replacing:

$\{u, v\} \rightarrow (u, v) \text{ and } (v, u)$

◆ 1. *vertex_count()*

? Prompt

What does *vertex_count()* do?

✓ Answer

Returns the **number of vertices** in the graph.

$\text{vertex_count}() \rightarrow |V|$

◆ 2. `vertices()`

? Prompt

What does `vertices()` return?

✓ Answer

Returns an **iterator over all vertices** in the graph.

✦ Used to:

- Traverse vertices
- Apply algorithms (DFS, BFS, etc.)

Diagram

Vertices: A, B, C

`vertices()` → [A, B, C]

◆ 3. `edge_count()`

? Prompt

What does `edge_count()` return?

✓ Answer

Returns the **number of edges** in the graph.

`edge_count()` → |E|

◆ 4. `edges()`

? Prompt

What does `edges()` return?

✓ Answer

Returns an **iterator of all edges** in the graph.

Diagram

Edges:

A — B

B — C

`edges()` $\rightarrow$ `[(A,B), (B,C)]`

◆ 5. `get_edge(u, v)`

? Prompt

What is `get_edge(u, v)`?

✓ Answer

Returns:

- The edge from vertex `u` to `v`, if it exists
- Otherwise returns `None`

✦ In undirected graphs:

`get_edge(u, v) == get_edge(v, u)`

Diagram

A — B

`get_edge(A,B)` $\rightarrow$ `Edge`

`get_edge(B,A)` $\rightarrow$ `Edge`

`get_edge(A,C)` $\rightarrow$ `None`

◆ 6. `degree(v, out=True)`

? Prompt

What does `degree(v)` mean?

✓ Answer

◆ Undirected Graph

`degree(v)` = number of incident edges

◆ Directed Graph

- `out=True` → out-degree
- `out=False` → in-degree

Diagram

A → B → C

↑

D

- `degree(B, out=True) = 1`
- `degree(B, out=False) = 2`

◆ 7. `incident_edges(v, out=True)`

? Prompt

What does `incident_edges(v)` return?

✓ Answer

Returns all edges **connected to vertex v**

- Undirected: all incident edges
- Directed:
 - `out=True` → outgoing edges
 - `out=False` → incoming edges

Diagram

$A \rightarrow B \rightarrow C$

`incident_edges(B, out=True) → (B → C)`

`incident_edges(B, out=False) → (A → B)`

◆ 8. `insert_vertex(x=None)`

? Prompt

What does `insert_vertex(x)` do?

✓ Answer

- Creates a new vertex
- Stores element x
- Adds it to the graph
- Returns the vertex reference

◆ 9. `insert_edge(u, v, x=None)`

? Prompt

What does `insert_edge(u, v, x)` do?

✓ Answer

- Creates an edge from u to v
- Stores element x
- Adds it to the graph
- Returns the edge reference

◆ 10. `remove_vertex(v)`

? Prompt

What happens when a vertex is removed?

✓ Answer

- Removes vertex v
- Removes **all edges incident to v**

✦ This prevents dangling edges.

◆ 11. `remove_edge(e)`

? Prompt

What does `remove_edge(e)` do?

✓ Answer

- Removes edge e
- Vertices remain intact

◆ Prompt

Why do we use a Graph ADT instead of direct implementation?

✓ Answer

- Separates **interface from implementation**
- Allows multiple representations:
 - Adjacency list
 - Adjacency matrix
- Makes algorithms reusable
- Cleaner, safer design

EDGE LIST STRUCTURE (Graph Representation)

◆ Prompt

What is an edge list structure in graphs?

✓ Answer

The **edge list structure** is the **simplest representation** of a graph.

A graph $G = (V, E)$ is stored as:

- $V \rightarrow$ an unordered list of all vertex objects
- $E \rightarrow$ an unordered list of all edge objects

Each **edge explicitly stores references to its endpoint vertices**, but **vertices do not store references to incident edges**.

✦ This representation is simple but **not efficient** for many graph operations.

◆ Prompt

How are vertices and edges stored in an edge list structure?

✓ Answer

- Vertices are stored in a list V
- Edges are stored in a list E
- Both V and E are implemented using doubly linked lists
(using `PositionalList` from Chapter 7)

✓ Edges know their vertices

✗ Vertices do **not** know their edges

◆ Prompt

What information does a vertex object store?

✓ Answer

Each **vertex object** v storing element x contains:

Element reference

- Stores user data (e.g., city name, airport code)
- Supports:
- `v.element()`
-

Position reference in list V

- Reference to the vertex's position in the vertex list
- Allows **O(1)** removal of the vertex from V

◆ Prompt

What information does an edge object store?

✓ Answer

Each **edge object e** storing element x contains:

Element reference

- Edge data (weight, label, cost)
- Supports:
- `e.element()`

Endpoint references

- References to the two endpoint vertices
- Supports:
- `e.endpoints()`
- `e.opposite(v)`

Position reference in list E

- Reference to its position in edge list
- Allows **O(1)** removal of the edge

◆ Prompt

How are Graph ADT operations supported in an edge list?

✓ Answer

Operation	How it Works	Time
<code>vertex_count()</code>	Return size of V	O(1)
<code>edge_count()</code>	Return size of E	O(1)
<code>vertices()</code>	Iterate over V	O(n)
<code>edges()</code>	Iterate over E	O(m)
<code>insert_vertex(x)</code>	Add to V list	O(1)
<code>insert_edge(u, v, x)</code>	Add to E list	O(1)
<code>remove_vertex(v)</code>	Remove v + scan E to remove incident edges	O(m)
<code>remove_edge(e)</code>	Remove from E list	O(1)
<code>get_edge(u, v)</code>	Scan all edges	O(m)
<code>degree(v)</code>	Scan all edges	O(m)
<code>incident_edges(v)</code>	Scan all edges	O(m)

✦ **m = number of edges, n = number of vertices**

◆ Prompt

Why is the edge list structure inefficient?

✓ Answer

Because:

- Vertices do **not** store references to their incident edges
- Many operations require scanning **all edges**

Examples:

- Finding neighbors of a vertex → must check every edge
- Checking adjacency → must scan entire edge list
- Degree calculation → must scan entire edge list

◆ Prompt

When is an edge list useful?

✓ Answer

✓ Very simple to implement

✓ Uses minimal memory

✓ Good for:

- Very small graphs
- Graph algorithms that naturally iterate over edges (e.g., Kruskal's MST)

◆ Prompt

What are the drawbacks of the edge list structure?

✗ Answer

✗ Inefficient adjacency queries

✗ Poor degree and incident edge performance

✗ Not suitable for dense graphs

✗ Most graph algorithms become slow

Comparison Snapshot

Representation	Best Use
Edge List	Simple graphs, edge-focused algorithms
Adjacency List	Sparse graphs
Adjacency Matrix	Dense graphs

Performance of the Edge List Structure

◆ Prompt

What is the space complexity of the edge list structure and why?

✓ Answer

The space complexity is $O(n + m)$, where:

- **n** = number of vertices
- **m** = number of edges

✦ Reason:

- Each **vertex object** uses **O(1)** space
- Each **edge object** uses **O(1)** space
- List **V** stores n vertices $\rightarrow O(n)$
- List **E** stores m edges $\rightarrow O(m)$

◆ Prompt

Why do `vertex_count()` and `edge_count()` run in **O(1)** time?

✓ Answer

Because:

- Vertices are stored in list **V**
- Edges are stored in list **E**
- The size of each list is directly accessible

✓ No traversal required

✓ Just return the stored size

🕒 Time Complexity

`vertex_count()` $\rightarrow O(1)$

`edge_count()` $\rightarrow O(1)$

◆ Prompt

Why do `vertices()` and `edges()` take **O(n)** and **O(m)** time?

✓ Answer

Because:

- `vertices()` iterates through all n vertices in **V**
- `edges()` iterates through all m edges in **E**

🕒 Time Complexity

`vertices()` $\rightarrow O(n)$

`edges()` $\rightarrow O(m)$

◆ Prompt

Why are `get_edge(u, v)`, `degree(v)`, and `incident_edges(v)` all $O(m)$?

✓ Answer

Because **vertices do NOT store references to incident edges**.

➡ The only way to answer these queries is to **scan the entire edge list E**.

🔍 Breakdown

`get_edge(u, v)`

- Must inspect every edge
- Check if its endpoints are (u, v)

`degree(v)`

- Count edges where v is an endpoint
- Requires scanning all edges

`incident_edges(v)`

- Collect all edges touching v
- Requires scanning all edges

◆ Prompt

Why are insertion operations $O(1)$?

✓ Answer

`insert_vertex(x)`

- Create vertex object
- Add it to list V
- Store its position

`insert_edge(u, v, x)`

- Create edge object
- Add it to list E

- Store its position

✓ No traversal

✓ Just constant-time list insertion

🕒 Time Complexity

`insert_vertex(x)` $\rightarrow O(1)$

`insert_edge(u, v, x)` $\rightarrow O(1)$

◆ Prompt

Why does `remove_edge(e)` run in $O(1)$ time?

✓ Answer

Each edge stores:

- A reference to its **position in list E**

So:

- We directly remove it using that position

✓ No search needed

✓ Constant-time removal

🕒 Time Complexity

`remove_edge(e)` $\rightarrow O(1)$

◆ Prompt

Why does `remove_vertex(v)` take $O(m)$ time?

✓ Answer

When removing vertex v :

1. All **incident edges must also be removed**
2. But vertex v does not know which edges touch it
3. We must **scan all edges in E**
4. Remove edges whose endpoints include v

Final Performance Summary Table

Operation	Running Time
<code>vertex_count()</code>	$O(1)$
<code>edge_count()</code>	$O(1)$
<code>vertices()</code>	$O(n)$
<code>edges()</code>	$O(m)$
<code>get_edge(u, v)</code>	$O(m)$
<code>degree(v)</code>	$O(m)$
<code>incident_edges(v)</code>	$O(m)$
<code>insert_vertex(x)</code>	$O(1)$
<code>insert_edge(u, v, x)</code>	$O(1)$
<code>remove_edge(e)</code>	$O(1)$
<code>remove_vertex(v)</code>	$O(m)$

✂ **Space Complexity:** $O(n + m)$

Adjacency List Structure

◆ Prompt

What is the adjacency list structure of a graph?

✓ Answer

An **adjacency list structure** represents a graph by:

- Storing all vertices in a primary collection V
- For each vertex v , storing a **secondary list** $I(v)$ that contains **only the edges incident to v**

This groups edges **by vertex**, unlike the edge list where all edges are in one list.

◆ Prompt

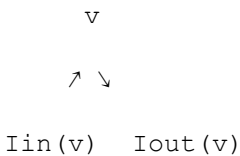
What is the incidence collection $I(v)$?

✓ Answer

$I(v)$ is the **list of all edges incident to vertex v** .

- In **undirected graphs**:
 $I(v)$ contains all edges connected to v
- In **directed graphs**:
 - $I_{out}(v) \rightarrow$ outgoing edges
 - $I_{in}(v) \rightarrow$ incoming edges

Directed Graph View



◆ Prompt

Why is this representation called an **adjacency list**?

✓ Answer

Traditionally:

- Each $I(v)$ is implemented as a **list**
- The list stores edges that connect v to **adjacent vertices**

Hence the name **adjacency list**.

◆ Prompt

How does the **adjacency list** store vertices and access $I(v)$ in $O(1)$ time?

✓ Answer

There are two common approaches:

✓ Option 1: Positional List

- Vertices stored in a positional list v
- Each vertex object stores a **direct reference to its $I(v)$**

✓ Option 2: Array-Based

- Vertices numbered 0 to $n-1$
- An array index gives direct access to $I(v)$

Both methods allow:

Access $I(v) \rightarrow O(1)$

Example Graph (From Figure 14.5)

◆ Prompt

How does an adjacency list represent an undirected graph?

Graph G

```
u — e — v
| \      |
g  f      h
|   \    |
w — z ----
```

Adjacency List Representation

V (Primary List):

$u \rightarrow v \rightarrow w \rightarrow z$

Incidence Lists:

$I(u): e, g, f$

$I(v): e, h$

$I(w): g, f$

$I(z): f, h$

Each edge appears **in two lists** (one for each endpoint).

◆ Prompt

Does the adjacency list duplicate edge objects?

✓ Answer

✗ No.

Each edge is represented by **one Edge object only**.

That Edge object:

- Stores references to its two endpoint vertices
- Is referenced in the incidence lists of both endpoints

◆ Prompt

Why does `incident_edges(v)` run in $O(\deg(v))$ time?

✓ Answer

Because:

- $I(v)$ contains **only edges incident to v**
- There are exactly $\deg(v)$ such edges

So iterating over $I(v)$:

`Time =  $O(\deg(v))$`

✓ This is **optimal**

✓ You cannot do better because you must report all $\deg(v)$ edges

◆ Prompt

What advantage does adjacency list have over edge list?

✓ Answer

Feature	Edge List	Adjacency List
<code>incident_edges(v)</code>	$O(m)$	$O(\deg(v))$
<code>degree(v)</code>	$O(m)$	$O(\deg(v))$
Local edge access	✗ Poor	✓ Excellent
Space	$O(n+m)$	$O(n+m)$

Adjacency list avoids scanning all edges.

◆ Prompt

What is the space complexity of an adjacency list?

✓ Answer

Space complexity is:

$$O(n + m)$$

- n vertices
- m edges
- Each edge stored once (with references in two lists)

Performance of the Adjacency List Structure

◆ Prompt

What is the space used by the adjacency list structure?

✓ Answer

- **Vertices:** $O(n) \rightarrow$ each vertex has a record in the primary list V
- **Edges:** $O(m) \rightarrow$ all edges are stored in the secondary lists $I(v)$

Note: For **undirected graphs**, each edge (u, v) appears in both $I(u)$ and $I(v)$, but this is still $O(m)$ because the constant factor does not change asymptotic complexity.

Total space complexity:

$O(n + m)$

◆ Prompt

What is the running time of `vertex_count()` and `edge_count()`?

✓ Answer

Operation	Running Time	Reason
<code>vertex_count()</code>	$O(1)$	Store the count as an attribute of the graph
<code>edge_count()</code>	$O(1)$	Maintain an auxiliary list E of all edges, or update a counter when edges are added/removed

◆ Prompt

How fast are `vertices()` and `edges()`?

✓ Answer

Operation	Running Time	Reason
<code>vertices()</code>	$O(n)$	Iterate through the primary list V of vertices
<code>edges()</code>	$O(m)$	Iterate through the secondary lists $I(v)$ (or use an auxiliary edge list E)

✎ Diagram of Iterating Edges via $I(v)$

Vertices V : u, v, w

$I(u)$: e_1, e_2

$I(v)$: e_1, e_3

$I(w)$: e_2, e_3

`edges()` → visit each secondary list and collect edges

$O(m)$ time

⚠ Be careful not to double-count undirected edges (u, v) .

◆ Prompt

How fast is `get_edge(u, v)` in an adjacency list?

✓ Answer

- Search **I(u)** or **I(v)** (whichever has fewer edges)
- Time complexity:

$O(\min(\deg(u), \deg(v)))$

This is much faster than edge lists ($O(m)$), especially for **sparse graphs**.

◆ Prompt

How fast can we compute `degree(v)`?

✓ Answer

- If **I(v)** supports size retrieval in $O(1)$, then:

$\text{degree}(v) = |\text{I}(v)| \rightarrow O(1)$

- Otherwise, iterate $\text{I}(v) \rightarrow O(\deg(v))$

◆ Prompt

What is the running time of `incident_edges(v)`?

✓ Answer

- Directly iterate over **I(v)**

$\text{Time} = O(\deg(v))$

- This is **optimal**, because we must report all incident edges.

◆ Prompt

What is the running time to insert vertices or edges?

✓ Answer

Operation	Running Time	Reason
<code>insert_vertex(x)</code>	O(1)	Add a new vertex to the primary list V
<code>insert_edge(u, v, x)</code>	O(1)	Add edge to I(u) and I(v) (or just Iout(u) / Iin(v) for directed graph)

⚠ Note: Adding to adjacency list is usually **constant time** because we maintain references.

◆ Prompt

What is the running time to remove an edge or vertex?

✓ Answer

Operation	Running Time	Reason
<code>remove_edge(e)</code>	$O(1)$	Use stored references to positions in $I(u)$ and $I(v)$
<code>remove_vertex(v)</code>	$O(\deg(v))$	Must remove all edges incident to v , which are stored in $I(v)$

⚠ Deleting a vertex is proportional to the number of its incident edges, not all edges.

📋 Summary Table (Table 14.3)

Operation	Running Time
<code>vertex_count()</code> , <code>edge_count()</code>	$O(1)$
<code>vertices()</code>	$O(n)$
<code>edges()</code>	$O(m)$
<code>get_edge(u, v)</code>	$O(\min(\deg(u), \deg(v)))$
<code>degree(v)</code>	$O(1)$
<code>incident_edges(v)</code>	$O(\deg(v))$
<code>insert_vertex(x)</code> , <code>insert_edge(u, v, x)</code>	$O(1)$
<code>remove_edge(e)</code>	$O(1)$
<code>remove_vertex(v)</code>	$O(\deg(v))$

- Space: $O(n + m)$

10 Visual Diagram of Adjacency List Performance

Graph:

```
u — e1 — v
|         |
e2       e3
|
w
```

Adjacency lists:

$I(u): e1, e2 \rightarrow \deg(u) = 2$

$I(v): e1, e3 \rightarrow \deg(v) = 2$

$I(w): e2, e3 \rightarrow \deg(w) = 2$

Operations:

- $\text{incident_edges}(u) \rightarrow e1, e2 \rightarrow O(\deg(u)) = O(2)$

- $\text{get_edge}(u, v) \rightarrow \text{search } I(u) \text{ or } I(v) \rightarrow O(\min(2, 2)) = O(2)$

- $\text{remove_vertex}(u) \rightarrow \text{remove } e1, e2 \rightarrow O(\deg(u)) = O(2)$

Adjacency Matrix Structure for Graphs

◆ Prompt

What is the adjacency matrix representation of a graph?

✓ Answer

- An **adjacency matrix** is a **2D array $\mathbf{A}$** of size $n \times n$, where n is the number of vertices.
- Each cell $A[i][j]$ represents the edge from vertex i to vertex j :
 - If edge (u, v) exists, $A[i][j]$ stores a **reference to the Edge object**.
 - If no edge exists, $A[i][j] = \text{None}$.
- For an **undirected graph**, the matrix is **symmetric**: $A[i][j] = A[j][i]$.

Key idea: The adjacency matrix allows **$O(1)$ worst-case access** to any edge.

◆ Prompt

How are vertices mapped to rows and columns in the adjacency matrix?

✓ Answer

- Vertices are numbered $0, 1, \dots, n-1$.
- Row i corresponds to vertex i (origin)
- Column j corresponds to vertex j (destination)
- The cell (i, j) stores the edge from vertex i to vertex j or **None**.

Example:

Vertices: $u, v, w, z \rightarrow$ indices 0, 1, 2, 3

◆ Prompt

Why use an adjacency matrix?

✓ Answer

1. **Constant-time edge lookup:**

- `get_edge(u, v) = A[i][j] → O(1)` worst-case

1. **Simple implementation for dense graphs** where most pairs of vertices have edges.
2. **Boolean adjacency matrix** can use 1 bit per edge if no extra edge data is needed.

◆ Prompt

What are the disadvantages of an adjacency matrix?

✓ Answer

High space usage: $O(n^2)$ for n vertices $\rightarrow$ inefficient for sparse graphs

Finding incident edges is slower:

- Must scan all n entries in row $i \rightarrow O(n)$
- Adjacency list can do it in $O(\deg(v))$

Adding/removing vertices is costly:

- Requires resizing the $n \times n$ array

Sparse graphs waste memory because most entries are `None`.

Comparison to adjacency list

Feature	Adjacency Matrix	Adjacency List
Space	$O(n^2)$	$O(n + m)$
Edge lookup	$O(1)$	$O(1)$ expected (dict)
Incident edges	$O(n)$	$O(\deg(v))$
Adding vertex	$O(n^2)$	$O(1)$

Dense graph	Efficient	Less efficient
Sparse graph	Inefficient	Efficient

Example Graph and Matrix

Graph:

Vertices: u=0, v=1, w=2, z=3

Edges: (u,v), (u,w), (v,w), (w,z)

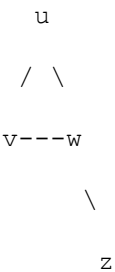
Adjacency Matrix:

	0(u)	1(v)	2(w)	3(z)
0(u)	None	e1	e2	None
1(v)	e1	None	e3	None
2(w)	e2	e3	None	e4
3(z)	None	None	e4	None

- e1 = Edge(u, v), e2 = Edge(u, w), e3 = Edge(v, w), e4 = Edge(w, z)
- Symmetric matrix for undirected graph

Visualization of Adjacency Matrix Structure

Graph G:



Adjacency Matrix (n=4):

	u	v	w	z
u	-	e1	e2	-
v	e1	-	e3	-
w	e2	e3	-	e4
z	-	-	e4	-

- Each edge is stored **once per endpoint** in the matrix
- Allows **direct access** to any edge without searching

Python Implementation of Graph ADT (Adjacency Map)

◆ Prompt

What is a Vertex and what information does it store?

✓ Answer

- Each `Vertex` stores an **element** (`x`), e.g., airport code, city name, or user ID.
- It has a method `element()` to retrieve the stored element.
- `hash()` is implemented so a `Vertex` can be used as a key in dictionaries or sets.

```
class Vertex:

    def __init__(self, x):

        self.element = x

    def element(self):

        return self.element

    def __hash__(self):

        return hash(id(self))
```

Key idea: Vertices are **lightweight objects**, and Python dictionaries can use them as keys.

◆ Prompt

What is an Edge and what methods does it provide?

✓ Answer

- Each `Edge` stores:
 - `_origin` → starting vertex
 - `_destination` → ending vertex

- o `_element` → optional data (weight, flight number, etc.)
- **Methods:**
 - o `endpoints()` → **returns** `(u, v)`
 - o `opposite(v)` → **returns** the other vertex on the edge
 - o `element()` → **returns** stored element
- Hash function allows `Edge` to be used in sets or dicts.

```
class Edge:

    def __init__(self, u, v, x):

        self.origin = u

        self.destination = v

        self.element = x

    def endpoints(self):

        return (self.origin, self.destination)

    def opposite(self, v):

        return self.destination if v is self.origin else self.origin

    def __hash__(self):

        return hash((self.origin, self.destination))
```

Key idea: Edges are **lightweight and self-contained**, holding only references to endpoints and optional element.

◆ Prompt

What is the main data structure for the graph?

✓ Answer

- **Adjacency map:**
 - o Top-level dictionary: `outgoing` maps `Vertex` → `Iout(v)` dictionary
 - o Optional `incoming` dictionary for directed graphs: `Vertex` → `Iin(v)` dictionary
- For **undirected graphs**, `incoming` is an alias of `outgoing`
- Each secondary dictionary `I(v)` maps **neighbor vertex** → **Edge instance**

```
class Graph:

    def __init__(self, directed=False):
```

```

self.outgoing = {}

self.incoming = {} if directed else self.outgoing

```

Key idea:

- Python dictionaries give **O(1) expected access** to incidence maps.
- Simplifies adjacency list implementation without maintaining separate V or E lists.

◆ Prompt

How is `is_directed()` implemented?

✓ Answer

```

def is_directed(self):

    return self.incoming is not self.outgoing

```

- If `incoming` and `outgoing` point to **different dictionaries**, the graph is directed.
- If they are **the same dictionary**, the graph is undirected.

◆ Prompt

Running time for `vertex_count()` and `edge_count()`?

✓ Answer

```

def vertex_count(self):

    return len(self.outgoing) # O(1)

def edge_count(self):

    total = sum(len(self.outgoing[v]) for v in self.outgoing)

    return total if self.is_directed() else total // 2 # O(n)

```

- **Vertex count:** $O(1)$ → just check dictionary size
- **Edge count:** $O(n)$ → iterate over all vertices' incidence maps
- Divide by 2 for undirected graphs to avoid double-counting edges

◆ Prompt

How are `vertices()` and `edges()` implemented?

✓ Answer

```
def vertices(self):  
    return self.outgoing.keys() # O(n)  
  
def edges(self):  
    result = set()  
    for secondary_map in self.outgoing.values():  
        result.update(secondary_map.values())  
    return result # O(n+m)
```

- **Vertices:** return dictionary keys
- **Edges:** union all secondary maps; use set to avoid duplicates for undirected graphs

◆ Prompt

How is `get_edge(u, v)` implemented?

✓ Answer

```
def get_edge(self, u, v):  
    return self.outgoing[u].get(v) # O(1) expected
```

- Looks in vertex `u`'s adjacency dictionary for neighbor `v`
- Returns `Edge` or `None` if not found

◆ Prompt

How are `degree(v)` and `incident_edges(v)` implemented?

✓ Answer

```
def degree(self, v, outgoing=True):  
    adj = self.outgoing if outgoing else self.incoming
```

```
return len(adj[v]) # O(1)
```

```
def incident_edges(self, v, outgoing=True):  
    adj = self.outgoing if outgoing else self.incoming  
    for edge in adj[v].values():  
        yield edge # O(deg(v))
```

- `degree(v)` → count of neighbors in adjacency dictionary
- `incident_edges(v)` → iterate through $I(v)$

⚡ Efficient: only depends on degree, not total edges.

◆ Prompt

How are `insert_vertex(x)` and `insert_edge(u,v,x)` implemented?

✓ Answer

```
def insert_vertex(self, x=None):  
    v = self.Vertex(x)  
    self.outgoing[v] = {}  
    if self.is_directed():  
        self.incoming[v] = {}  
    return v
```

```
def insert_edge(self, u, v, x=None):  
    e = self.Edge(u, v, x)  
    self.outgoing[u][v] = e  
    self.incoming[v][u] = e  
    return e
```

- Create new `Vertex` or `Edge` objects
- Update adjacency dictionaries

- Time: $O(1)$ expected

10 Diagram of Graph Representation in Python

Vertices: u, v, w

```
Graph.outgoing = {  
    u: {v: e1, w: e2},  
    v: {u: e1, w: e3},  
    w: {u: e2, v: e3}  
}
```

```
Graph.incoming = Graph.outgoing (undirected)
```

Edges:

```
e1 = Edge(u, v)
```

```
e2 = Edge(u, w)
```

```
e3 = Edge(v, w)
```

Operations:

```
- get_edge(u,v) → Graph.outgoing[u][v] = e1
```

```
- degree(u) → len(Graph.outgoing[u]) = 2
```

```
- incident_edges(u) → iterate Graph.outgoing[u].values() → [e1, e2]
```